

# CSE 4616

## Lab 01: Introduction to OMNeT++ and INET

Duration: 1 hour 30 minutes — In-Lab Assessment

**Tools:** OMNeT++ and INET Framework

---

### Objectives

---

- Understand what OMNeT++ and INET are and why they are used.
- Learn basic OMNeT++ components: NED files, modules, gates, and connections.
- Understand the structure and parameters of a NED file for a mixed wired and wireless network.
- Understand the structure and parameters of an `omnetpp.ini` file for configuring simulation behaviour.
- Run a prebuilt INET example in the OMNeT++ IDE (Qtenv) and inspect results.

### 1. Short Theory

---

#### 1.1 What is OMNeT++?

OMNeT++ is a discrete event simulation environment used to model communication networks, queuing networks, and distributed systems. It provides an IDE (graphical) where you can write simple network descriptions (NED), implement behaviour in C++ if needed, run the simulation in a visual environment (Qtenv), and collect results.

#### 1.2 What is INET?

INET is a large open-source model library/framework built on OMNeT++. It provides prebuilt, realistic protocol stacks (Ethernet, IP, TCP/UDP, wireless PHY/MAC, routing, and lots of applications). Using INET saves you from implementing low-level protocols and lets you focus on experiments.

#### 1.3 Key OMNeT++ Concepts

- **NED files:** Describe the network topology (modules and how they connect).
- **Modules:** Building blocks. A *simple module* contains behaviour (in C++). A *compound module* contains submodules and connections.
- **Gates and connections:** Modules communicate through gates, connected by channels (wired or wireless links).
- **omnetpp.ini:** Main configuration file specifying which network to run, parameter values, and runtime choices.
- **Qtenv:** The graphical runtime where you can see packet animations.

- **Wildcard rules:** `**` matches any module path; `*` matches one level. More specific parameter keys always override less specific ones.

## 2. Before the Lab

---

- OMNeT++ installed (use the official installer for your OS).
- INET framework downloaded and built inside OMNeT++ (import INET into the IDE and build; many INET examples appear in the Project Explorer).
- Familiarize yourself with: `File` → `New` → `OMNeT++ Project`, and how to open `omnetpp.ini` and NED files.
- If you have trouble installing, follow the documentation provided in the classroom.

## 3. Part A: Run a Ready-Made INET Example

---

1. Open OMNeT++ IDE.
2. In the Project Explorer open the INET examples folder (e.g. `inet/examples/wireless/wiredandwirelesshostswithap`). You should see NED networks and `omnetpp.ini` files.
3. Right-click on the example's `omnetpp.ini` and choose **Run As** → **OMNeT++ Simulation**.
4. In Qtenv you will see nodes and links. Press the green play button to let the simulation run (packets animate) or use step controls to single-step events.
5. Open the console/Status window to observe INET app outputs (RTT, packet delivery). The simulation writes scalar/vector output files into the `results` directory.

## 4. Part B: NED File Explanation

---

**File:** `WiredAndWirelessHostsWithAP.ned`

The NED file is the **blueprint** of your network. It defines what devices exist and how they are physically connected. Below is the complete file followed by a detailed explanation.

```
1 //
2 // SPDX-License-Identifier: LGPL-3.0-or-later
3 //
4
5 package inet.examples.wireless.wiredandwirelesshostswithap;
6
7 import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator
8     ;
9 import inet.node.ethernet.Eth100M;
10 import inet.node.inet.Router;
11 import inet.node.inet.StandardHost;
12 import inet.node.inet.WirelessHost;
```

```

12 import inet.node.wireless.AccessPoint;
13 import inet.physicallayer.wireless.ieee80211.packetlevel.
    Ieee80211ScalarRadioMedium;
14 import inet.visualizer.contract.IIntegratedVisualizer;
15
16 network WiredAndWirelessHostsWithAP
17 {
18     submodules:
19         visualizer: <default(firstAvailableOrEmpty("
IntegratedCanvasVisualizer"))>
20             like IIntegratedVisualizer if typename != "" {
21                 parameters:
22                     @display("p=100,300;is=s");
23             }
24         configurator: Ipv4NetworkConfigurator {
25             parameters:
26                 assignDisjunctSubnetAddresses = false;
27                 @display("p=100,100;is=s");
28         }
29         radioMedium: Ieee80211ScalarRadioMedium {
30             parameters:
31                 @display("p=100,200;is=s");
32         }
33         wirelessHost1: WirelessHost {
34             parameters:
35                 @display("p=62,69");
36         }
37         wiredHost1: StandardHost {
38             parameters:
39                 @display("p=296,134");
40         }
41         wiredHost2: StandardHost {
42             parameters:
43                 @display("p=412,70");
44         }
45         router: Router {
46             parameters:
47                 @display("p=296,69");
48         }
49         accessPoint: AccessPoint {
50             parameters:
51                 @display("p=205,69");
52         }
53     connections:
54         accessPoint.ethg++ <--> Eth100M <--> router.ethg++;
55         wiredHost1.ethg++ <--> Eth100M <--> accessPoint.ethg++;
56         wiredHost2.ethg++ <--> Eth100M <--> router.ethg++;
57 }

```

Listing 1: WiredAndWirelessHostsWithAP.ned

## 4.1 Package and Import Lines

- **package inet.examples.wireless.wiredandwirelesshostswithap;** — Declares the location of this file within the INET project folder structure, preventing name clashes.
- **import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;** — Brings in the automatic IP address assigner and route configurator.
- **import inet.node.ethernet.Eth100M;** — Imports the 100 Mbps Ethernet cable type used for wired connections.
- **import inet.node.inet.Router;** — A standard network router that forwards packets between different network segments.
- **import inet.node.inet.StandardHost;** — A regular wired computer/host connected via Ethernet.
- **import inet.node.inet.WirelessHost;** — A device that connects via Wi-Fi (has a built-in wireless radio).
- **import inet.node.wireless.AccessPoint;** — A Wi-Fi Access Point bridging the wired and wireless parts of the network.
- **import inet.physicallayer.wireless.ieee80211.packetlevel.Ieee80211ScalarRadioMedium;** — The simulated *air* through which Wi-Fi signals propagate. Without this, no wireless communication is possible.
- **import inet.visualizer.contract.IIntegratedVisualizer;** — Provides visual overlays (packet movement, wireless ranges) during simulation.

## 4.2 Submodules Explanation

## 4.3 Connections and Network Topology

The connections block defines the physical cables. The syntax is:

```
moduleA.gate++ <--> ChannelType <--> moduleB.gate++
```

<--> denotes a **bidirectional** connection; ++ automatically assigns the next available port. All three cables use Eth100M (100 Mbps Ethernet).

### Resulting Topology:

```
wirelessHost1
  |
  (Wi-Fi)
  |
accessPoint ===== router ===== wiredHost2
  ||
  (Eth100M)
  ||
wiredHost1
```

| Module        | Type                       | Purpose                                                                                  |
|---------------|----------------------------|------------------------------------------------------------------------------------------|
| visualizer    | IntegratedVisualizer       | Optional animation overlays in QtEnv                                                     |
| configurator  | Ipv4NetworkConfigurator    | Auto-assigns IPs; assignDisjunctSubnetAddresses=false keeps all hosts on the same subnet |
| radioMedium   | Ieee80211ScalarRadioMedium | The virtual medium carrying Wi-Fi signals                                                |
| wirelessHost1 | WirelessHost               | Wi-Fi enabled device; connects to accessPoint wirelessly                                 |
| wiredHost1    | StandardHost               | Wired PC; connected to accessPoint via Ethernet                                          |
| wiredHost2    | StandardHost               | Wired PC; connected to router via Ethernet                                               |
| router        | Router                     | Forwards packets between network segments                                                |
| accessPoint   | AccessPoint                | Bridges wired Ethernet and wireless radio                                                |

Table 1: Submodule summary for WiredAndWirelessHostsWithAP

## 5. Part C: INI File Explanation

**File:** omnetpp.ini

The omnetpp.ini file is the **instruction manual** for your simulation. It tells OMNeT++ what to simulate, who talks to whom, and how traffic behaves.

```

1 [General]
2 network = WiredAndWirelessHostsWithAP
3 sim-time-limit = 400s
4
5 **.addDefaultRoutes = false
6
7 **.*Host*.numApps = 3
8
9 **.*Host*.app[0].typename = "UdpEchoApp"
10 **.*Host*.app[0].localPort = 1000
11
12 **.*Host*.app[*].typename = "UdpBasicApp"
13 **.[wiredHost1.app][1].destAddresses = "wirelessHost1"
14 **.[wiredHost1.app][2].destAddresses = "wiredHost2"
15 **.[wiredHost2.app][1].destAddresses = "wirelessHost1"
16 **.[wiredHost2.app][2].destAddresses = "wiredHost1"
17 **.[wirelessHost1.app][1].destAddresses = "wiredHost1"
18 **.[wirelessHost1.app][2].destAddresses = "wiredHost2"

```

```

19 **.*Host*.app[1..].destPort = 1000
20 **.*Host*.app[1..].messageLength = 100B
21 **.*Host*.app[1..].sendInterval = 1s
22 **.*Host*.app[1..].stopTime = 300s
23
24 **.initialZ = 0m

```

Listing 2: omnetpp.ini

## 5.1 Line-by-Line Explanation

- **[General]** — A section header. Everything below applies globally to all simulation runs.
- **network = WiredAndWirelessHostsWithAP** — Selects the network defined in the NED file. The name must match exactly.
- **sim-time-limit = 400s** — The simulation automatically stops at 400 simulated seconds.
- **\*\*.\*addDefaultRoutes = false** — Disables the automatic default gateway entry. Hosts still receive direct/specific routes from `Ipv4NetworkConfigurator`, so communication is unaffected within this small closed network. A default route is only needed to reach external or unknown networks.
- **\*\*.\*Host\*.numApps = 3** — Every host whose name contains “Host” runs **3 applications** simultaneously (`app[0]`, `app[1]`, `app[2]`).
- **app[0].typename = "UdpEchoApp"** — The first app on every host is a *UdpEchoApp*: it listens for incoming packets and immediately echoes them back to the sender.
- **app[0].localPort = 1000** — The echo app listens on port 1000. Any packet arriving at port 1000 is handled by `app[0]`.
- **app[\*].typename = "UdpBasicApp"** — Sets a *default* type for all app indices. Because `app[0]` already has a more specific rule (`UdpEchoApp`), OMNeT++ applies `UdpBasicApp` only to `app[1]` and `app[2]`. **Specificity, not order, determines priority.**

### 5.1.1 Traffic Destination Map

Each host’s two sending apps (`app[1]` and `app[2]`) are directed to the other two hosts, forming a fully connected communication triangle:

- **app[1..].destPort = 1000** — All sending apps target port 1000, where the `UdpEchoApp` is listening. The `1..` syntax means “index 1 and above”.
- **app[1..].messageLength = 100B** — Each packet is 100 bytes in size.

| Source Host   | App    | Destination   |
|---------------|--------|---------------|
| wiredHost1    | app[1] | wirelessHost1 |
| wiredHost1    | app[2] | wiredHost2    |
| wiredHost2    | app[1] | wirelessHost1 |
| wiredHost2    | app[2] | wiredHost1    |
| wirelessHost1 | app[1] | wiredHost1    |
| wirelessHost1 | app[2] | wiredHost2    |

Table 2: Traffic destination configuration

- **app[1..].sendInterval = 1s** — One packet is sent every simulated second per app; each host generates 2 packets/second in total.
- **app[1..].stopTime = 300s** — Sending stops at 300 s. The simulation runs until 400 s, leaving 100 quiet seconds at the end.
- **\*\*.initialZ = 0m** — Places all nodes at height 0 (ground level) for 3D visualization. No effect on simulation logic.

## 5.2 INI File Precedence Rule

**Key Rule:** OMNeT++ uses **specificity-based priority**, not file order. A parameter targeted at app[0] is always more specific than app[\*], so app[0] retains UdpEchoApp regardless of where the lines appear in the file.

app[0]            ← Exact index (highest priority)  
app[1..2]        ← Index range  
app[\*]            ← Wildcard (lowest priority, acts as default)

## 5.3 Packet Lifecycle Example

1. At  $t = 1$  s, wiredHost1's app[1] sends a 100 B UDP packet to wirelessHost1:1000.
2. The packet travels through the network (via the access point over Wi-Fi).
3. wirelessHost1's app[0] (UdpEchoApp) receives it on port 1000.
4. The echo app sends the packet back to wiredHost1.
5. This repeats every second until  $t = 300$  s.

## 6. Part D: Summary Tables

---

| Parameter           | Value / Meaning                        |
|---------------------|----------------------------------------|
| Network             | WiredAndWirelessHostsWithAP            |
| Simulation Duration | 400 seconds                            |
| Hosts               | wiredHost1, wiredHost2, wirelessHost1  |
| Apps per Host       | 3 (1 echo server + 2 senders)          |
| Traffic Pattern     | Every host sends to the other two      |
| Packet Size         | 100 Bytes                              |
| Send Rate           | 1 packet/second per sender             |
| Traffic Active      | 0 s – 300 s                            |
| Default Routes      | Disabled (specific routes still added) |
| Wireless Standard   | IEEE 802.11 (Wi-Fi)                    |
| Wired Link Speed    | 100 Mbps (Eth100M)                     |

Table 3: Simulation configuration summary

## 7. Debugging Tips

---

- **No packets visible / instant simulation end:** Check `sim-time-limit` in `omnetpp.ini` and ensure `startTime` of apps is within the limit.
- **Hosts cannot communicate:** Ensure `Ipv4NetworkConfigurator` is present in the NED file and that `assignDisjunctSubnetAddresses` is set appropriately.
- **Wrong import or module name:** INET versions move module classes between packages. Search for the module name in your INET `src` tree using the IDE search.
- **Build errors:** Ensure INET is built first and your project references it via `Project` → `Properties` → `Project References`.
- **Parameter not applied:** Verify the wildcard path matches the actual module hierarchy. Use the OMNeT++ inspector (right-click a module) to view all available parameters.